

Chapter 4

Responsive Web Design

INT150 Web Technologies (2/2025)

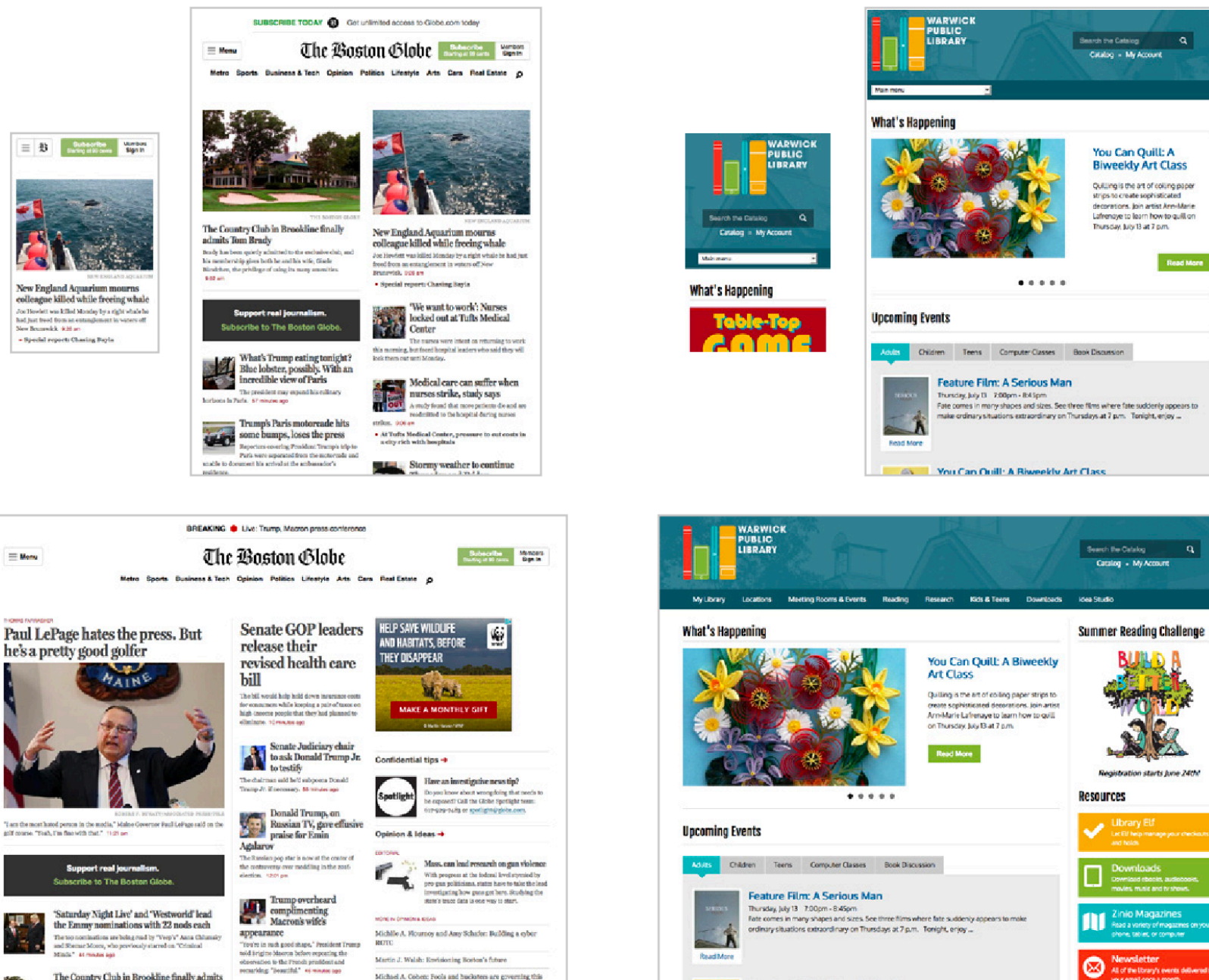
Sanit Sirisawatvatana

Introduction to RWD

- **RWD** is a design and production approach that allows a web page to adapt to the screen it is viewed on.
- The same HTML source is styled different based on the width of screen.
- It ensures mobile users get the same content as other users with a presentation that optimizes usability.
- The term was coined by *Ethan macotte* in 2010 in his article "Responsive Web Design" for A List Apart.

“The core principle is that all devices get the same HTML source, located at the same URL, but different styles are applied base on the viewport size to rearrange components and optimize usability.”

Examples of responsive sites



The Boston Globe
bostonglobe.com

Warwick Public Library
warwicklibrary.org

Setting the Viewport

- The **Viewport** is the user's visible area of a web page.
- The **viewport** varies with the device, and will be smaller on a mobile phone than on a computer screen.
- To fit standard websites onto small screens, mobile browsers render the page on a canvas (**viewport**) and shrink that viewport down to fit the width of the screen (**device width**).
- To make the viewport the same size as the device width:

```
<meta name="viewport" content="width=device-width, initial-scale="1">
```

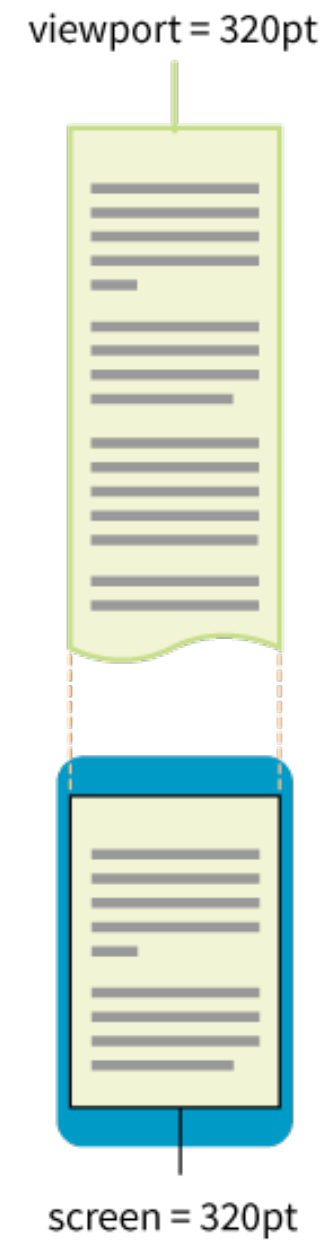
- The **meta** element, which goes in the **head** of the HTML document, tells the browser to set the width of the viewport equal to the width of the device screen. The **initial-scale** value sets the zoom level to **1 (100%)**.

By default, the viewport shrinks to the size of the screen.



With the viewport meta tag, the viewport is created at the same size as the screen.

```
<meta name="viewport"
      content="width=device-width,
              initial-scale=1">
```



The components of RWD

Ethan marcotte defines the three core components of RWD as follows:

- A flexible grid (Fluid Layouts)
 - Elements are sized so they squeeze or flow into available browser space.
- Flexible images
 - Images and other embedded media scale to fit their containing elements.
- CSS media queries
 - These are a way to test for browser features such as viewport width and deliver styles accordingly.

Flexible Grids (Fluid Layout)

- **Fluid layouts** are the best approach for designing for a wide range of screen widths.
- In a **fluid layout**, the page area and its grid resize proportionally to fill the available width of the screen or window.
- For flexibility, use:
 - **fr** and **minmax()** units in CSS grid layouts
 - **flex** property settings in Flexbox
 - **percentage values** for horizontal measurements so elements remain proportional at varying sizes

Flexible Grids (Fluid Layout)

Fluid layouts fill the viewport proportionally.



w3c.org

Fixed layouts stay the same size and may get cut off or leave extra space.



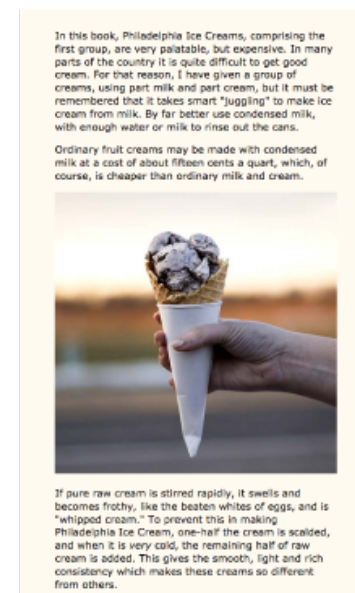
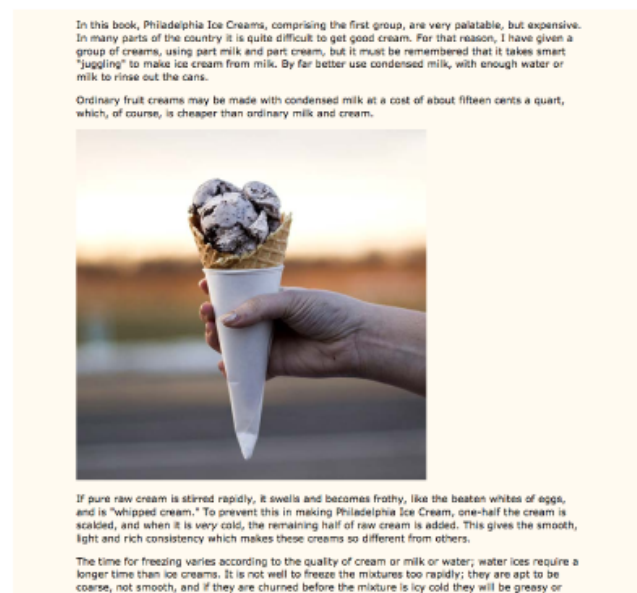
kexp.org

Making Images Flexible

- To make images scale down to fit the size of their container:

```
img { max-width: 100% ; } /* not the same as img { width: 100% ; } */
```

- When the layout gets smaller, the images in it scale down to fit the width of their respective container.
- If the container is larger than the image, the image does not scale larger; it stops at 100% of its original size.
- If you do set the `width` attribute, be sure the `height` attribute is set to `auto`; otherwise, the image won't scale proportionately.



Media Queries

- A **media query** is a special rule that tests for certain conditions before applying the style it contains:

```
@media type and (feature : value ){  
    /* styles for browsers that meet this criteria */  
}
```

- A few query examples:
 - Is it on screen or printed?
 - Is the browser at least a certain width?
 - Is the orientation landscape or portrait?

Media Queries Example

```
@media screen and (orientation: landscape) {  
  body {  
    background: skyblue;  
  }  
}  
@media screen and (orientation: portrait) {  
  body {  
    background: coral;  
  }  
}
```



When the viewport is in portrait mode, the background color is "coral."



When the viewport is in landscape mode, the background color is "skyblue."

Media Types

- The first part of a media query identifies the **media type**.
- The current media types are **all**, **print**, **screen**, and **speech**.
- **Media types** were introduced in CSS2.
- If you are designing for **screen**, the media type is **optional**, so you can omit it.
- Example:

```
@media print {
```

```
    /*print-specific styles between the brackets */
```

```
}
```

Media feature queries

- [CSS3](#) introduced media [feature queries](#) that test for particular feature of a viewport or device.
- The most common feature to test for is the viewport width ([min-width](#) and [max-width](#)).
- Example:

```
h1 { font-family: Georgia, serif ; }
```

```
@media screen and (min-width: 40em) {
```

```
  h1 { font-family: 'Lobster', cursive ; }
```

```
}
```

- Browser displays headline fonts in fancy cursive font only when the viewport is 40em or wider. Viewports that do not match the query use a simple serif face.

Media features

Feature	Description
width	The width of the display area (viewport). Also min-width and max-width .
height	The height of the display area (viewport). Also min-height and max-height .
orientation	Whether the device is in portrait or landscape orientation.
aspect-ratio	Ratio of the viewport's width divided by height (width/height). Example: aspect-ratio: 16/9 .
color	The bit depth of the display; for example, color: 8 tests for whether the device has at least 8-bit color.
color-index	The number of colors in the color lookup table.
monochrome	The number of bits per pixel in a monochrome device.
resolution	The density of pixels in the device. This is increasingly relevant for detecting high-resolution displays.
scan	Whether a tv media type uses progressive or interlace scanning. (Does not accept min-/max- prefixes.)
grid	Whether the device uses a grid-based display, such as a terminal window. (Does not accept min-/max- prefixes.)

Media features

Deprecated features

The following features have been deprecated in Media Queries Level 4 Working Draft and are discouraged from use.

device-width	The width of the device's rendering surface (the whole screen). (Deprecated in favor of width .)
device-height	The height of the device's rendering surface (the whole screen). (Deprecated in favor of height .)
device-aspect-ratio	Ratio of the whole screen's (rendering surface) width to height. (Deprecated in favor of aspect-ratio .)

New in Media Queries Level 4

update-frequency	How quickly (if at all) the output device modifies the appearance of the content.
overflow-block	How the device handles content that overflows the viewport along the block axis.
overflow-inline	Whether the content that overflows the viewport along the inline axis can be scrolled.
color-gamut	The approximate range of colors that are supported by the user agent and output device.
pointer	Whether the primary input mechanism is a pointing device and how accurate it is.
hover	Whether the input mechanism allows the user to hover over elements.
any-pointer	Whether any available input mechanism is a pointing device, and how accurate it is.
any-hover	Whether any available input mechanism allows hovering.

Using Media Queries

- Use **media queries** within a style sheet or to conditionally load external style sheets.
- **Media queries** may not be used within inline style.
{ do not permit }
- When you use **media queries** within a style sheet, the order of rules is very important.
- The rules later in the style sheet override the rules that come before them.
- Your media query needs to come after any rules with the same declaration.

Using Media Queries

- The strategy is to specify the baseline styles that serve as a default, and then override specific rules as needed to optimize for alternate viewing environments.
- In RWD, the best practice is to set up styles for small screens and browsers that don't support media queries, and then introduce styles for increasingly larger screens later in the style sheet.

"Mobile-First Design"

Baseline styles

Tablet styles

Desktop styles

With external style sheets

- For large or complicated sites, developers may choose to put styles for different devices into separate style sheets and call in the whole .css file when certain conditions are met.
- One method is to use the media attribute in the link element to conditionally load separate .css files.

- Example:

```
<head>  
  <link rel="stylesheet" href="styles.css">  
  <link rel="stylesheet" href="2column-style.css"  
      media="screen and (min-width: 1024px)" >  
</head>
```

- Example

```
<style>  
  @import url("/default-styles.css") ;  
  @import url("/wide-styles.css) screen and (min-width: 1024px) ; /* no "media" word*/  
</style>
```

Breakpoints

- A **breakpoint** is the point at which we use a media query to introduce a style change.

Example

```
h1 { font-family: Georgia, serif ; }  
@media screen and (min-width: 800px) {  
  h1 { font-family: 'Lobster', cursive ; }  
}
```

The 800 pixels is the "breakpoint" at which the font family for <h1> should be used.

- **Best practice is to create breakpoints for individual parts of the page** rather than the entire page at one.
- **Module-Based breakpoints**
 - **Designers generally create the narrow-screen version first** and then change components as needs as the screen grows larger.
- **Em-based breakpoints** introduce layout change proportional to font size (ems) instead of pixels.
 - Pixel-based media queries don't adapt if the user changes their font size setting, which people do in order to be able to read the page more easily.
 - But em-based media queries respond to the size of the text, keeping the layout of the page in proportion.

Example

```
/* Changing everything at once, even if some parts don't need to. */
```

```
@media (max-width: 768px) {  
  .navbar { flex-direction: column; }  
  .stock-card { width: 100%; }  
  .chart { display: none; }  
}
```

```
/* 1. Navbar: Switch to mobile menu when elements start to crowd at 500px */
```

```
@media (max-width: 500px) {  
  .navbar-links { display: none; }  
  .hamburger-menu { display: block; }  
}
```

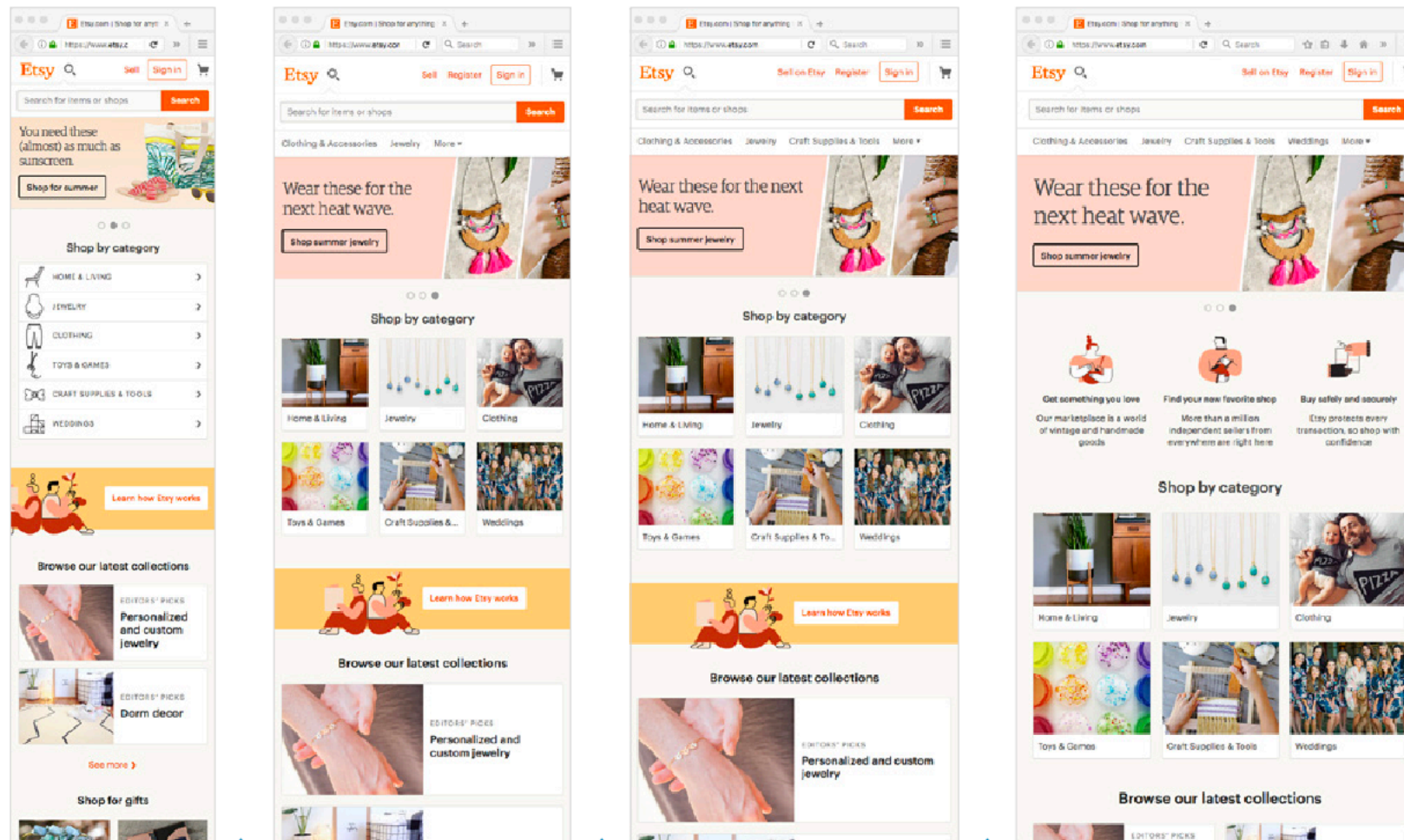
```
/* 2. Stock Card: Stack cards vertically if the screen width is less than 400px */
```

```
@media (max-width: 400px) {  
  .stock-card-container { flex-direction: column; }  
}
```

```
/* 3. Chart: Hide legends on screens narrower than 600px to save space */
```

```
@media (max-width: 600px) {  
  .chart-legend { display: none; }  
}
```

A series of breakpoints used by Etsy's responsive site (2017)

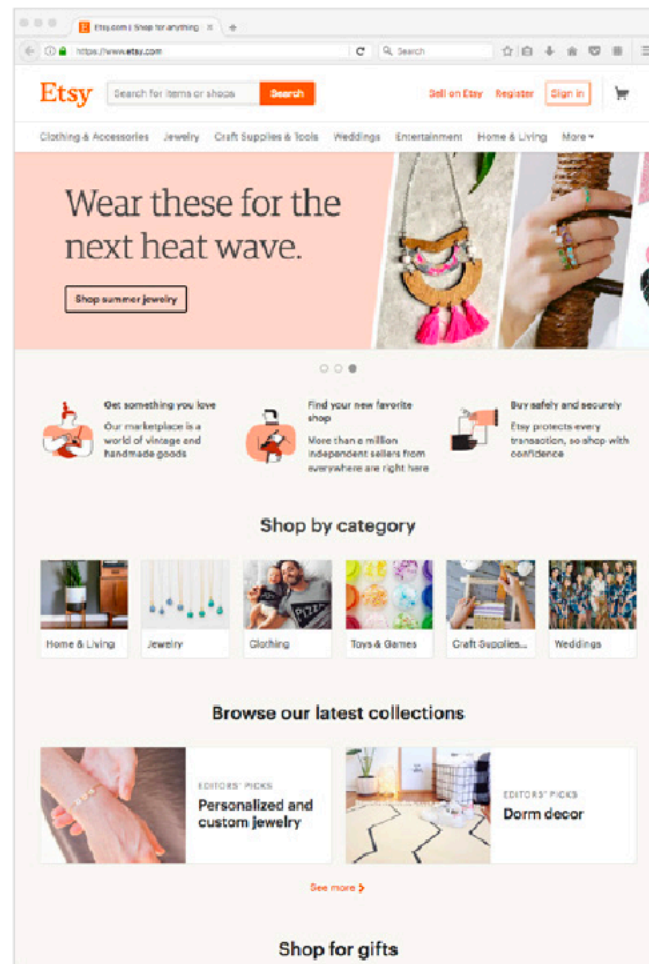


At the **480-pixel breakpoint**, the category navigation changes from a list to photos. “Register” is added to the top navigation bar.

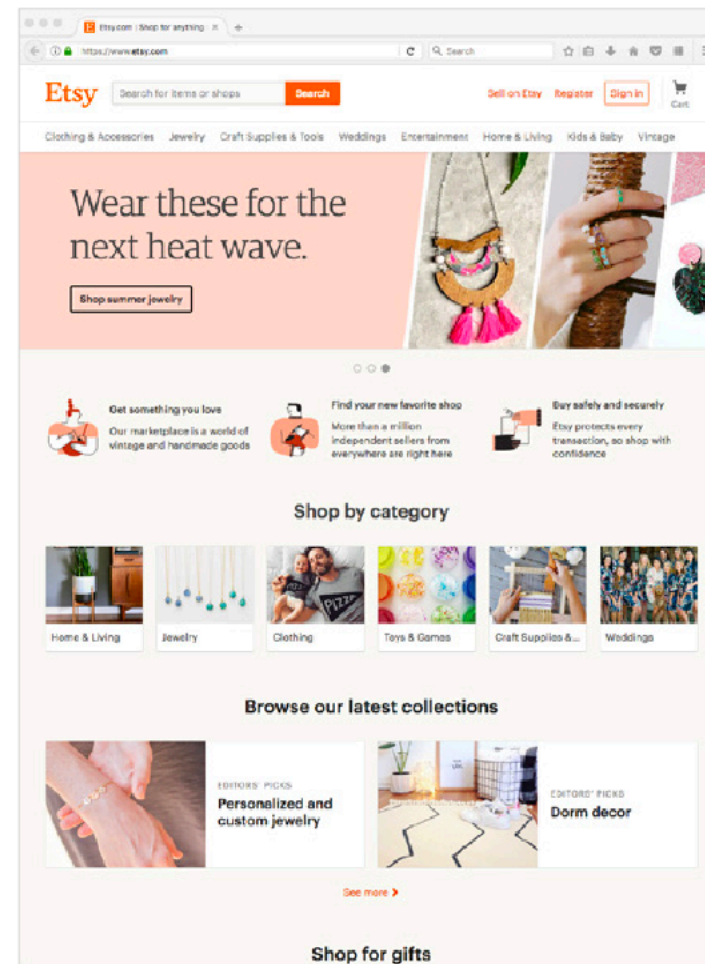
At **501 pixels**, “Sell” becomes “Sell on Etsy” (a very subtle adjustment). You can also see more links in the navigation bar under the search field.

At **640 pixels**, the “How Etsy Works” images and messages move above the categories. In smaller views, they were accessible via the “Learn how Etsy works” link in a yellow bar.

A series of breakpoints used by Etsy's responsive site (2017)



At **901 pixels**, the search input form moves into the top header.



At **981 pixels**, the word “Cart” appears under the shopping cart icon. We now see the full list of navigation options in the header (no “More” link). At this point, the layout expands to fill larger windows until it reaches its maximum width of 1400 pixels. Then margins add space equally on the left and right to keep the layout centered.

Designing Responsively

- You need to pay attention to the following aspects of the page in a responsive design.
 - Content hierarchy
 - Layout
 - Typography
 - Navigation
 - Images
 - Special content like table and form

Content Hierarchy

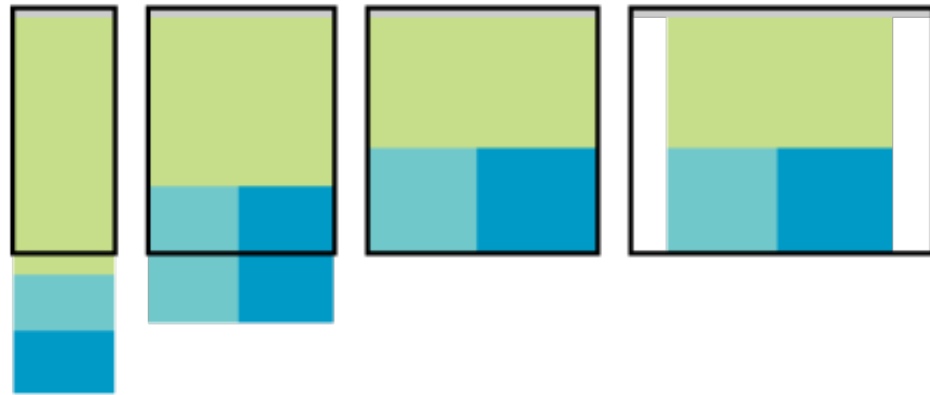
- [Content is king of the web](#), so it is critical that content is carefully considered and organized before any code gets written.
- [Organization and hierarchy](#) across various views of the site are a [primary concern](#), which a particular focus on the small-screen experience.
 - start with [potential contents](#) and then provide what is [most useful](#) and [important](#) for all browsing experiences.
 - Once you know what the content modules are, you can begin deciding in [what order they appear](#) on various screen sizes.
- You should strive for [content parity](#) (equality) across devices.
 - Everyone should have access to the same information, even if it's not in exactly the same place.

Layout

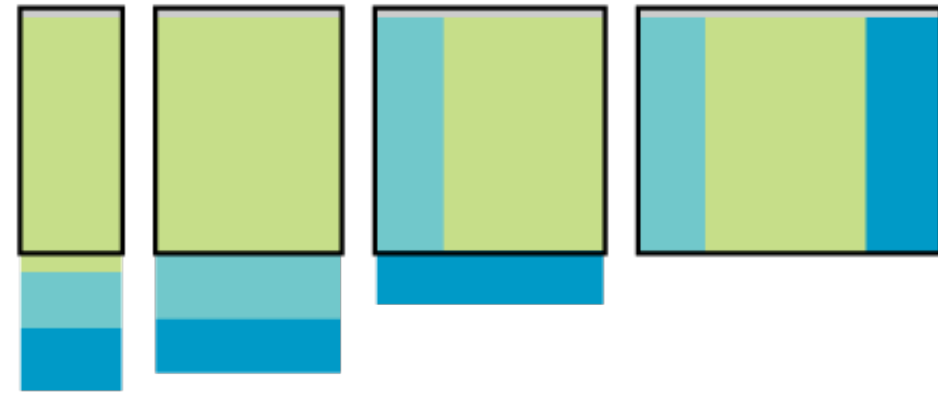
- RWD is based on fluid layout.
- The most common approach is to start with one column on small devices and add columns as space is available.
- The design process involve a certain amount of switching between views and making decisions about what happens along the way.
- The optimum width of a text element should be based on a line length of 45 to 75 characters including spaces. If line are much longer, it's time to make changes to the layout (introducing a breakpoint):
 - Increasing the margins
 - Introducing an additional column
 - Increasing the font size

Responsive Layout Patterns

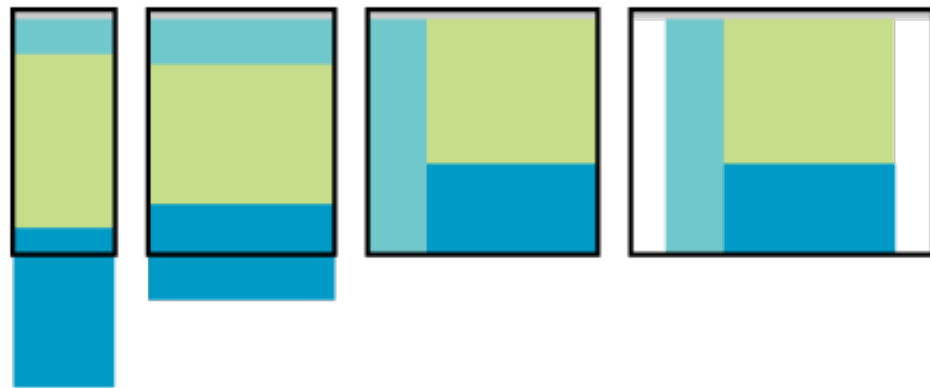
Mostly fluid



Column drop



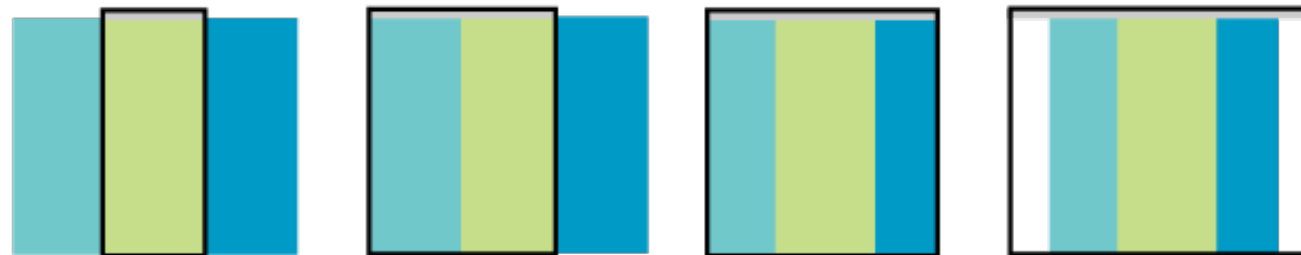
Layout shifter



Tiny tweaks



Off canvas



Responsive Layout Patterns

Mostly fluid

This pattern uses a single-column layout for small screens, and another fluid layout that covers medium and large screens, with a maximum width set to prevent it from becoming too wide. It generally requires less work than other solutions.

Column drop

This solution shifts between one-, two-, and three-column layouts based on available space. When there isn't room for extra columns, the sidebar columns drop below the other columns until everything is stacked vertically in the one-column view.

Layout shifter

If you want to get really fancy, you can completely reinvent the layout for a variety of screen sizes. Although expressive and potentially cool, it is not necessary. In general, you can solve the problem of fitting your content to multiple environments without going overboard.

Tiny tweaks

Some sites use a single-column layout and make tweaks to type, spacing, and images to make it work across a range of device sizes.

Off canvas

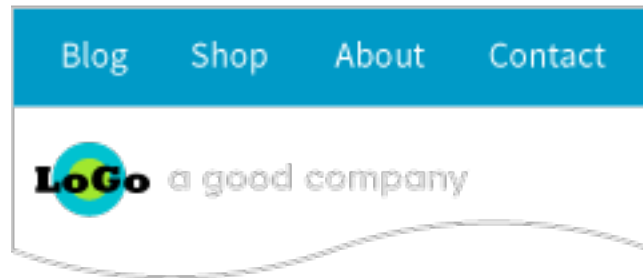
As an alternative to stacking content vertically on small screens, you may choose to use an “off-canvas” solution. In this pattern, a page component is located just out of sight on the left or right of the screen and flies into view when requested. A bit of the main content screen remains visible on the edge to orient users as to the relationship of moving parts. This was made popular by Facebook, wherein Favorites and Settings were placed on a panel that slid in from the left when users clicked a menu icon.

Typography

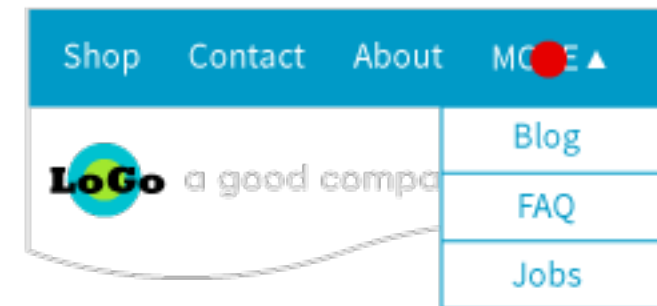
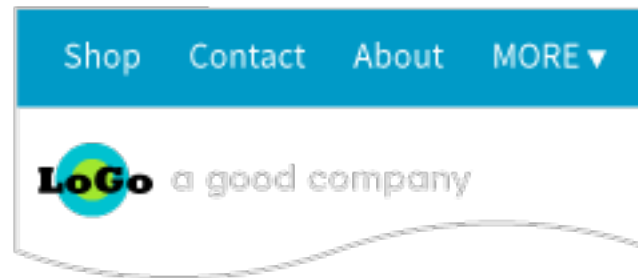
- Typography should be optimized for different screen sizes in order to keep it legible and pleasant to read.
- Avoid complicated font styles on small screens.
- Set font size to keep line length between 45 to 75 characters.
- Line height should generally be larger on larger screens.
 - On average, line height should be 1.5 (line-height property)
 - On small screen, it should be 1.2 to 1.5
 - On larger screen, it should be 1.4 to 1.6
- Left and right margin can increase on larger screen.

Navigation Options

Top navigation



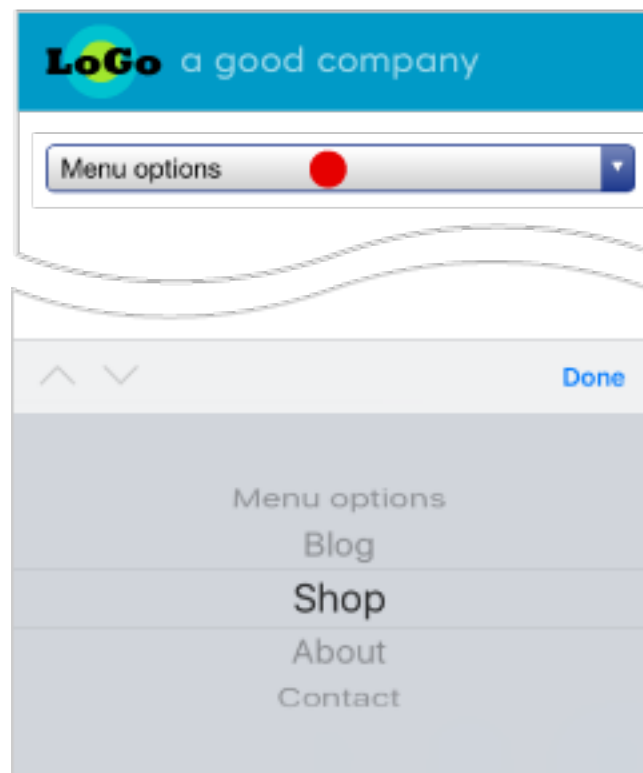
Priority +



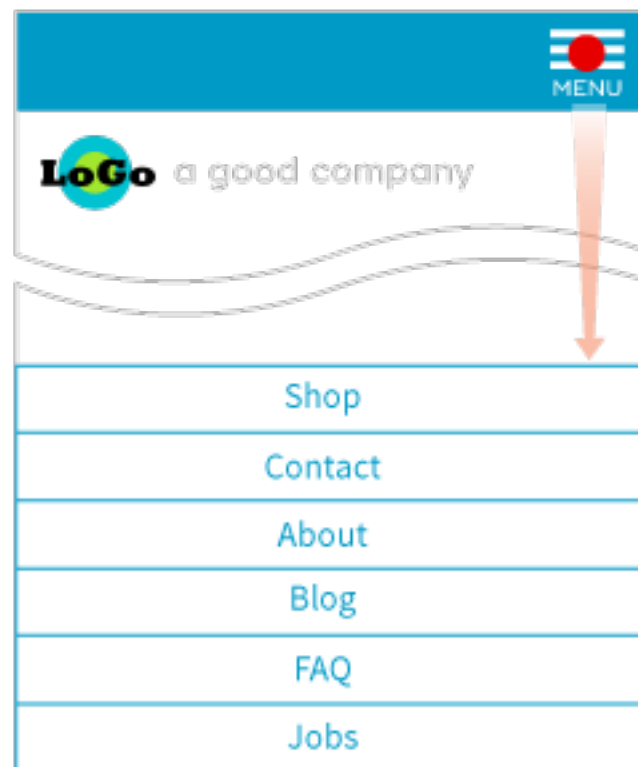
KEY



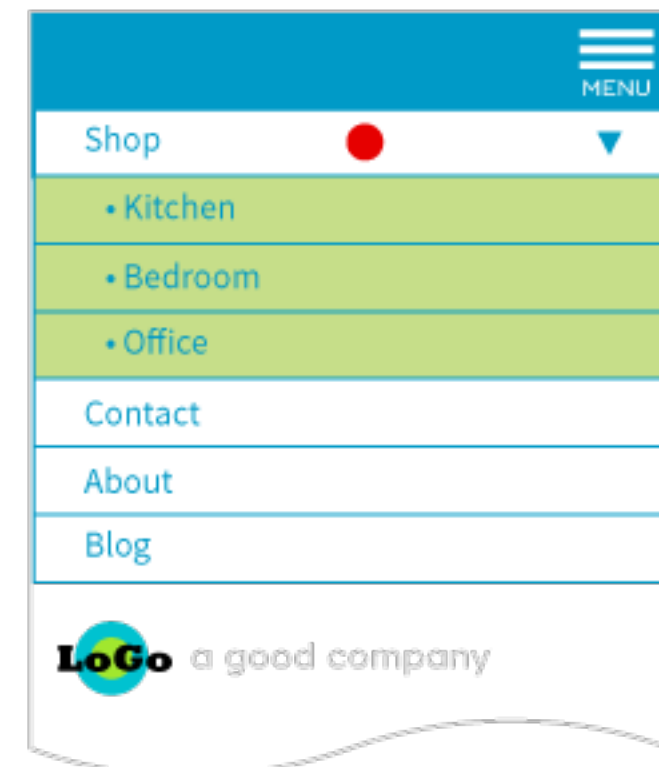
Select menu



Link to footer menu



Accordion sub-navigation



Navigation Options

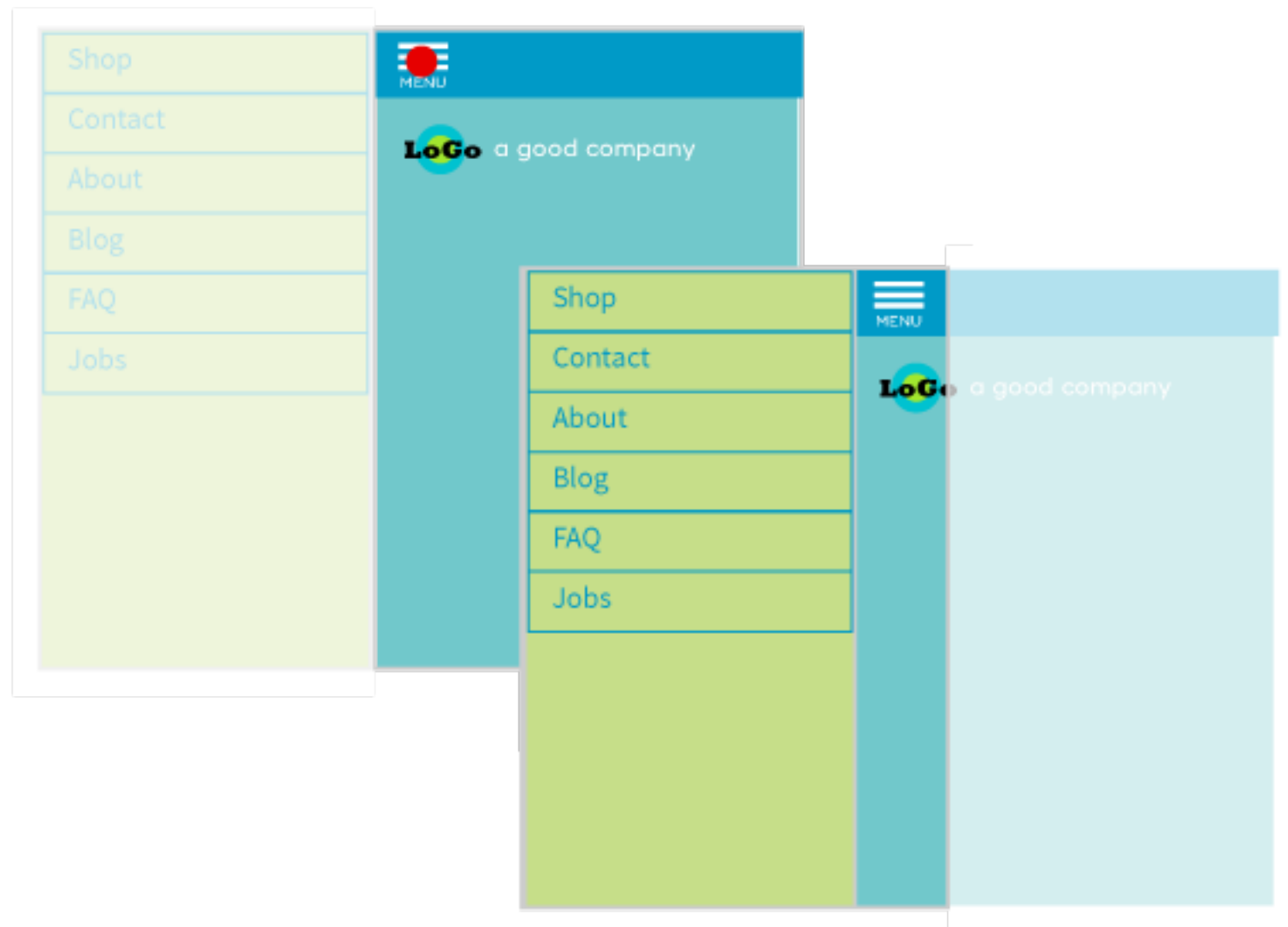
Overlay toggle (covers top of screen)



Push toggle (pushes content down)



Off-canvas/fly-in



Navigation Options

Top navigation

If your site has just a few navigation links, they may fit just fine in one or two rows at the top of the screen.

Priority +

In this pattern, the most important navigation links appear in a line across the top of the screen alongside a More link that exposes additional options. The pros are that the primary links are in plain view, and the number of links shown can increase as the device width increases. The cons include the difficulty of determining which links are worthy of the prime small-screen real estate.

Select menu

For a medium list of links, some sites use a **select** input form element. Tapping the menu opens the list of options using the select menu UI of the operating system, such as a scrolling list of links at the bottom of the screen or on an overlay. The advantage is that it is compact, but on the downside, forms aren't typically used for navigation, and the menu may be overlooked.

Link to footer menu

One straightforward approach places a Menu link at the top of the page that links to the full navigation located at the bottom of the page. The risk with this pattern is that it may be disorienting to users who suddenly find themselves at the bottom of the scroll.

Accordion sub-navigation

When there are a lot of navigation choices with sub-navigation menus, the small-screen solution becomes more challenging, particularly when you can't hover to get more options as you can with a mouse. Accordions that expand when you tap a small arrow icon are commonly used to reveal and hide sub-navigation. They may even be nested several levels deep. To avoid nesting navigation in accordion submenus, some sites simply link to separate landing pages that contain a list of the sub-navigation for that section.

Push and overlay toggles

In toggle navigation, the navigation is hidden but expands downward when the menu link is tapped. It may push the main content down below it (push toggle) or slide down in front of the content (overlay toggle).

Off-canvas/fly-in

This popular pattern puts the navigation in an off-screen panel to the left or right of the main content that slides into view when you tap the menu icon.

Images

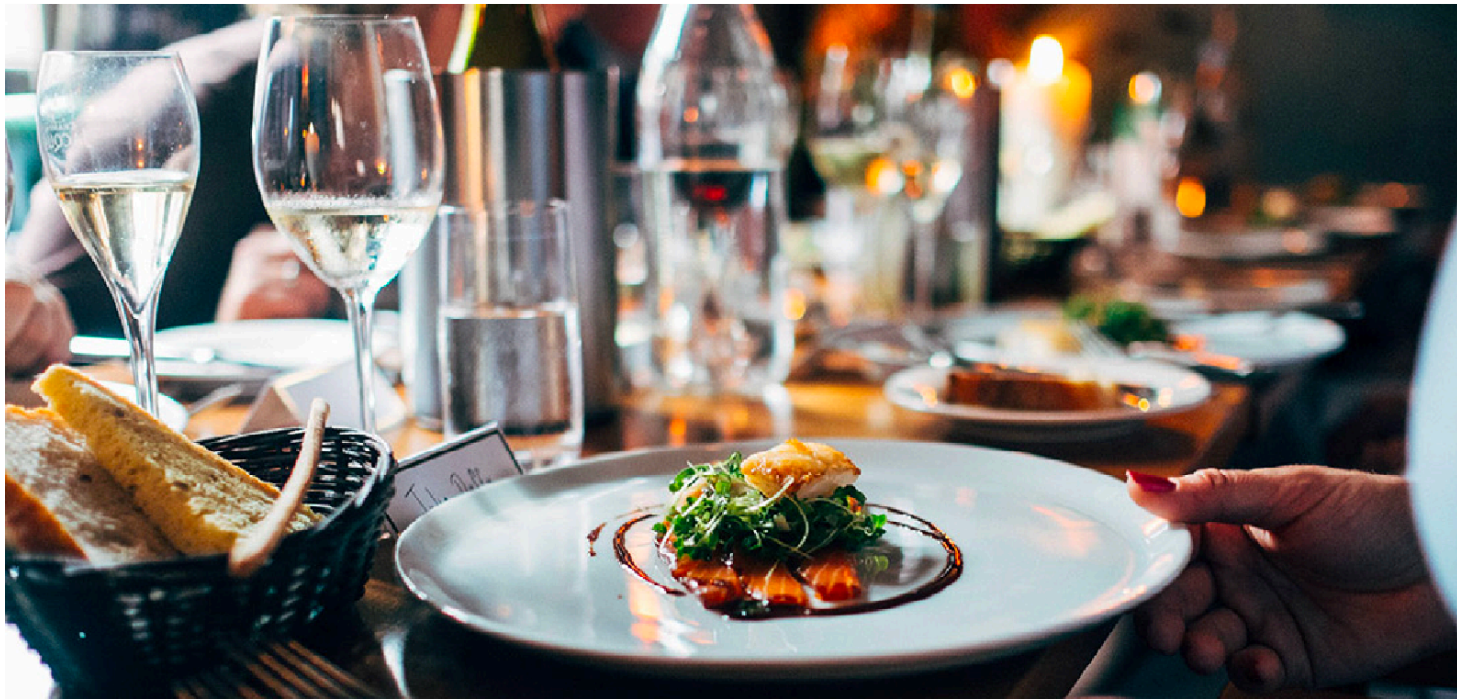
- Use responsive image markup techniques.
- Serve the smallest file size version by default to prevent unnecessary data downloads.
- Make sure important image detail isn't at smaller sizes. Allow zooming or provide a cropped version.
- Avoid text in graphics or provide alternative versions with larger type for small screens.

Responsive Image Markup

```

```

Art Direction (picture element)



That dinner looks delicious on desktop browsers.
(1280px wide)



Detail is lost when the full image is
shrunk down on small devices.
(300px wide)



Cropping to the most important detail
may make better sense.
(300px wide)

```
<picture>  
  <source media="(min-width: 1024px)" srcset="icecream-large.jpg">  
  <source media="(min-width: 760px)" srcset="icecream-medium.jpg">  
    
</picture>
```

Alternative Image Formats

```
<picture>  
  <source type="image/webp" srcset="pizza.webp">  
  <source type="image/jxr" srcset="pizza.jxr">  
    
</picture>
```

Special Content

- Forms
 - Flexbox is a great tool for adding flexibility and conditional wrapping to form fields and their labels. Form inputs will be use with fingertips, not mouse pointer and by making labels tappable to select an input.
- Tables
 - One of the greatest challenges in small-screen design is how to deal with large data tables (many types of tables, many solution).
- Interactive elements
 - Scale videos proportionally.
 - Explore alternatives that work better on small screens (for example, links to a map app instead of just providing a small embedded map).